

Energieeffiziente Echtzeitübertragung über CAN und CAN-FD

Adaptives Monitoring und bedarfsgesteuertes Scheduling
zur simultanen Gewährleistung von Echtzeitgarantien
und minimaler Leistungsaufnahme

Stephan Epp

24. Juni 2026

Zusammenfassung

Controller Area Network (CAN) und dessen Erweiterung CAN-FD (Flexible Data Rate) sind die dominierenden Feldbussysteme in der Fahrzeug- und Industrieautomatisierung. Die vorliegende Arbeit untersucht, wie durch ein kontinuierliches Busmonitoring und ein bedarfsgesteuertes, adaptives Scheduling die Übertragung jederzeit in Echtzeit gewährleistet werden kann, während gleichzeitig der Energiebedarf aller Busteilnehmer auf ein nachweisbares Minimum reduziert wird. Wir formalisieren das Problem, beweisen die Existenz und Optimalität eines adaptiven Scheduling-Verfahrens und illustrieren die theoretischen Ergebnisse anhand quantitativer Simulationen.

Inhaltsverzeichnis

1	Einführung	3
1.1	Beitrag dieser Arbeit	3
1.2	Aufbau der Arbeit	3
2	Grundlagen	3
2.1	CAN und CAN-FD	3
2.2	Echtzeittheorie	4
2.3	Energiemodell	5
3	Problemformalisierung	5
3.1	Systemmodell	5
3.2	Optimierungsproblem	5
4	Adaptives Monitoring und Scheduling-Verfahren	5
4.1	Monitoring-Architektur	5
4.2	Algorithmus	6
4.2.1	Formaler Pseudocode	6
4.2.2	Ausführliche Beschreibung des Algorithmus	7
4.2.3	Beweis der Korrektheit	8

4.2.4	Beweis der Laufzeit	9
4.2.5	Python-Implementierung	10
4.3	Modi-Definition für CAN und CAN-FD	11
5	Formale Beweise	11
5.1	Zulässigkeit des adaptiven Scheduling	11
5.2	Energieminimalität	12
5.3	Monitoring-Overhead	13
6	Quantitative Analyse und Simulationsergebnisse	14
6.1	Energieverbrauch in Abhängigkeit der Bus-Auslastung	14
6.2	Worst-Case-Latenz	14
6.3	Monitoring-Overhead und Energieeinsparung	15
6.4	Rahmeneffizienz	16
6.5	Adaptives Scheduling in Echtzeit	16
6.6	Leistungsaufnahme verschiedener Betriebsmodi	17
7	Zusammenfassung	18
8	Ausblick	18
8.1	Erweiterung auf CAN-XL und zukünftige Bussysteme	18
8.2	Maschinelles Lernen zur Lastprädiktion	18
8.3	Hardware-in-the-Loop-Validierung	19
8.4	Integration in AUTOSAR Adaptive	19
8.5	Formale Verifikation mit TLA+ und UPPAAL	19
8.6	Erweiterung auf Ethernet-basierte Fahrzeugnetzwerke	19
8.7	Sicherheitsaspekte des Monitoring-Kanals	19
8.8	Dynamische Lastbalancierung in Multi-Bus-Architekturen	20
8.9	Einfluss von EMV und Leitungsparasiten	20
8.10	Standardisierungsbestrebungen	20
	Literaturverzeichnis	20

1 Einführung

Eingebettete Systeme in Fahrzeugen, Industriesteuerungen und der Robotik kommunizieren überwiegend über serielle Feldbusse. CAN nach ISO 11898 [1] und sein Nachfolger CAN-FD nach ISO 11898-1:2015 [2] dominieren diesen Bereich seit Jahrzehnten. Gleichzeitig steigt der Druck, den Energieverbrauch in eingebetteten Systemen zu senken – sei es aus ökologischen Gründen, aus Gründen der Wärmeabfuhr oder aus batterietechnischen Anforderungen in Elektrofahrzeugen [4].

Die zentrale Herausforderung lautet: *Kann man CAN und CAN-FD so betreiben, dass (a) alle Nachrichten ihre Echtzeitdeadline einhalten und (b) der Energieverbrauch des Bussystems auf ein Minimum reduziert wird?*

Diese Ziele scheinen zunächst widersprüchlich: Echtzeitgarantien erfordern ausreichend Busbandbreite und damit Aktivität der Transceiver; Energieeinsparung erfordert Schlafzustände und niedrige Übertragungsfrequenzen. Die vorliegende Arbeit zeigt, dass beide Ziele durch ein intelligentes, bedarfsgesteuertes Monitoring gemeinsam erreichbar sind.

1.1 Beitrag dieser Arbeit

- Formales Modell für energiebewusstes Echtzeit-Scheduling auf CAN/CAN-FD.
- Beweis der Zulässigkeit des adaptiven Scheduling-Verfahrens (Theorem 3).
- Beweis der Energieminimalität unter Echtzeitbedingungen (Theorem 4).
- Quantitative Analyse der Laufzeit und des Overheads des Monitoring-Systems.
- Matplotlib-basierte Visualisierungen der theoretischen Ergebnisse.

1.2 Aufbau der Arbeit

Abschnitt 2 legt die Grundlagen zu CAN, CAN-FD, Echtzeittheorie und Energiemodellen dar. Abschnitt 3 formalisiert das Problem. Abschnitt 4 beschreibt das adaptive Monitoring- und Scheduling-Verfahren. Abschnitt 5 beweist die zentralen Sätze. Abschnitt 6 präsentiert die quantitativen Ergebnisse. Abschnitt 7 gibt einen ausführlichen Ausblick.

2 Grundlagen

2.1 CAN und CAN-FD

Definition 1 (CAN-Rahmen). Ein CAN-Rahmen besteht aus Start-of-Frame (SOF), Arbitrierungsfeld (11 oder 29 Bit Identifier), Steuerfeld (DLC, 4 Bit), Datenfeld (0–8 Byte), CRC-Feld (15 Bit), Acknowledge-Slot (1 Bit) und End-of-Frame (7 Bit). Die nominale Bitrate beträgt bis zu $f_{\text{CAN}} = 1 \text{ Mbit/s}$.

Definition 2 (CAN-FD-Rahmen). Ein CAN-FD-Rahmen erweitert das Datenfeld auf bis zu 64 Byte und unterteilt die Übertragung in zwei Phasen:

$$\begin{aligned} \text{Arbitrierungsphase: } f_{\text{arb}} &\leq 1 \text{ Mbit/s,} \\ \text{Datenphase: } f_{\text{data}} &\leq 8 \text{ Mbit/s.} \end{aligned}$$

Das Bit Rate Switch (BRS) Bit signalisiert den Übergang zwischen den Phasen.

Definition 3 (Bit-Stuffing). CAN verwendet Non-Return-to-Zero (NRZ)-Codierung mit Bit-Stuffing: Nach fünf identischen aufeinanderfolgenden Bits wird automatisch ein komplementäres Bit eingefügt. Im Worst Case erhöht dies die Rahmengröße um den Faktor $\frac{6}{5}$.

Lemma 1 (Maximale CAN-Rahmengröße). Die maximale Übertragungszeit $T_{\max}$ eines CAN-Rahmens mit d Datenbytes bei Bitrate f_b beträgt:

$$T_{\max}(d, f_b) = \frac{1}{f_b} \cdot \left\lfloor \frac{6}{5}(34 + 8d) \right\rfloor + \frac{3}{f_b},$$

wobei 34 Bit der feste Overhead (ohne Daten) und $8d$ die Nutzdatenbits sind.

Beweis. Der CAN-Datenrahmen umfasst ohne Bit-Stuffing die Felder: SOF (1) + Arbitrierung (12) + Steuerfeld (6) + Daten ($8d$) + CRC (16) + ACK (2) + EOF (7) = $44 + 8d$ Bit. Die Felder SOF, Arbitrierung, Steuerfeld und Daten (insgesamt $34 + 8d$ Bit) unterliegen dem Bit-Stuffing; CRC, ACK und EOF (10 Bit) sind davon ausgenommen. Mit dem Worst-Case-Faktor $\frac{6}{5}$ ergibt sich:

$$N_{\text{bits}} = \left\lfloor \frac{6}{5}(34 + 8d) \right\rfloor + 10.$$

Division durch f_b liefert $T_{\max}$. □

Beispiel 1. Für $d = 8$ Byte und $f_b = 1$ Mbit/s:

$$N_{\text{bits}} = \left\lfloor \frac{6}{5} \cdot 98 \right\rfloor + 10 = 117 + 10 = 127 \text{ Bit}, \quad T_{\max} = 127 \mu\text{s}.$$

2.2 Echtzeittheorie

Definition 4 (Periodische Echtzeit-Task). Eine periodische Echtzeit-Task τ_i ist durch das Tripel $\tau_i = (C_i, T_i, D_i)$ beschrieben, wobei C_i die Worst-Case-Übertragungszeit (WCET), T_i die Periode und $D_i \leq T_i$ die relative Deadline bezeichnen.

Definition 5 (Schedulierbarkeit). Eine Menge von Tasks $\{\tau_1, \dots, \tau_n\}$ heißt *schedulierbar*, wenn jeder Task vor Ablauf seiner Deadline abgeschlossen werden kann. Für EDF (Earliest Deadline First) auf einem Bus mit Kapazität 1 gilt das klassische Auslastungskriterium:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1.$$

Definition 6 (Response-Time-Analyse). Die Antwortzeit R_i eines CAN-Tasks τ_i mit Priorität p_i erfüllt die Fixpunktgleichung [5]:

$$R_i^{(k+1)} = C_i + J_i + \sum_{j \in \text{hp}(i)} \left\lceil \frac{R_i^{(k)} + J_j}{T_j} \right\rceil C_j,$$

wobei J_i der maximale Jitter von τ_i ist und $\text{hp}(i)$ die Menge der Tasks mit höherer Priorität als τ_i bezeichnet.

2.3 Energiemodell

Definition 7 (Energiemodell für CAN-Transceiver). Sei P_{tx} die Sendeleistung, P_{rx} die Empfangsleistung, P_{stby} die Standby-Leistung und P_{sleep} die Schlafleistung. Der Energieverbrauch eines Transceivers im Zeitintervall $[0, T]$ beträgt:

$$E(T) = P_{\text{tx}} \cdot t_{\text{tx}} + P_{\text{rx}} \cdot t_{\text{rx}} + P_{\text{stby}} \cdot t_{\text{stby}} + P_{\text{sleep}} \cdot t_{\text{sleep}},$$

mit $t_{\text{tx}} + t_{\text{rx}} + t_{\text{stby}} + t_{\text{sleep}} = T$.

Typische Werte für einen NXP TJA1044-Transceiver [6]: $P_{\text{tx}} = 75 \text{ mW}$, $P_{\text{rx}} = 50 \text{ mW}$, $P_{\text{stby}} = 2,5 \text{ mW}$, $P_{\text{sleep}} = 0,5 \text{ mW}$.

3 Problemformalisierung

3.1 Systemmodell

Wir betrachten ein CAN- bzw. CAN-FD-Netzwerk mit N Knoten $\{n_1, \dots, n_N\}$, die über einen gemeinsamen Bus kommunizieren. Jeder Knoten n_k besitzt eine Menge periodischer Tasks $\Gamma_k = \{\tau_{k,1}, \dots, \tau_{k,m_k}\}$. Die Gesamtmenge aller Tasks sei $\Gamma = \bigcup_{k=1}^N \Gamma_k$.

Definition 8 (Bus-Auslastung). Die aktuelle Bus-Auslastung zum Zeitpunkt t ist definiert als:

$$\rho(t) = \frac{\text{belegte Buszeit im Fenster } [t - W, t]}{W},$$

wobei W die Beobachtungsfensterbreite ist.

Definition 9 (Adaptiver Betriebsmodus). Ein adaptiver Betriebsmodus $\mathcal{M} = \{M_1, M_2, \dots, M_K\}$ ist eine geordnete Menge von Konfigurationen der Form $(f_{b,k}, P_{\text{mode},k}, \rho_{\min,k}, \rho_{\max,k})$, wobei $f_{b,k}$ die Bitrate, $P_{\text{mode},k}$ die Leistungsaufnahme und $[\rho_{\min,k}, \rho_{\max,k})$ das Auslastungsintervall für Modus M_k bezeichnet.

3.2 Optimierungsproblem

Definition 10 (Energie-Echtzeit-Optimierungsproblem). Gegeben sei ein Bus-System mit Taskmenge Γ . Gesucht ist ein Scheduling-Verfahren Π , das:

$$\text{minimiere} \quad E_{\Pi}(T) = \sum_{k=1}^N E_k(T) \tag{P1}$$

$$\text{unter} \quad \forall \tau_i \in \Gamma : R_i \leq D_i \tag{C1}$$

$$\forall t : \rho(t) \leq 1 \tag{C2}$$

4 Adaptives Monitoring und Scheduling-Verfahren

4.1 Monitoring-Architektur

Das vorgeschlagene Verfahren **ACES** (*Adaptive CAN Energy Scheduler*) besteht aus drei Komponenten:

1. **Bus-Monitor (BM)**: Misst $\rho(t)$ über gleitendes Fenster W .

2. **Mode-Controller (MC):** Wählt Betriebsmodus M_k gemäß $\rho(t)$.
3. **Deadline-Checker (DC):** Verifiziert $R_i \leq D_i$ für alle i vor Moduswechsel.

4.2 Algorithmus

4.2.1 Formaler Pseudocode

Der ACES-Algorithmus ist in Pseudocode 1 vollständig spezifiziert. Er arbeitet als reaktive Kontrollschleife, die in jeder Iteration genau vier wohldefinierte Schritte ausführt: Messen, Auswählen, Prüfen, Schalten.

Listing 1: ACES – Formaler Pseudocode

```

1 Algorithmus ACES(Gamma, W, M, Delta)
2   Eingabe:
3     Gamma          -- Menge periodischer CAN-Tasks {(C_i, T_i,
4       D_i)}
5     W              -- Breite des gleitenden Beobachtungsfensters
6       (ms)
7     M = {M_1,...,M_K} -- geordnete Modusmenge, aufsteigend nach
8       Leistung
9     Delta          -- Monitoring-Intervall (ms)
10
11   Ausgabe:
12   Kontinuierliche Steuerung des CAN-Transceivers; kein
13   Rueckgabewert.
14
15   Initialisierung:
16     current_mode <- M_1          // Modus mit minimaler
17     Leistung
18     rho          <- 0           // Bus-Auslastung (
19     Initialwert)
20
21   Hauptschleife (laeuft unbegrenzt):
22     // ---- Schritt 1: Bus-Auslastung messen ----
23     rho <- BUS_MONITOR(W)
24     // Zaehle belegte Buszeit im Fenster [t-W, t]; dividiere
25     durch W
26
27     // ---- Schritt 2: Kandidat-Modus bestimmen ----
28     candidate <- SELECT_MODE(M, rho)
29     // Waehle M_k mit rho_min,k <= rho < rho_max,k und
30     minimalem P_mode,k
31
32     // ---- Schritt 3: Schedulierbarkeit pruefen ----
33     feasible <- DEADLINE_CHECK(Gamma, candidate.bitrate)
34     // Berechne fuer jedes tau_i die Fixpunktiteration der
35     // Response-Time-Analyse; pruefe R_i <= D_i fuer alle i
36
37     // ---- Schritt 4: Moduswechsel (nur wenn zulaessig und
38     noetig) ----
39     if feasible and candidate != current_mode then

```

```

31     SWITCH_MODE(current_mode, candidate)
32     // Sende BRS-Signal (CAN-FD) oder passe Prescaler an (CAN
33     // );
34     // warte t_switch <= 10 us auf physikalische
35     // Stabilisierung
36     current_mode <- candidate
37 end if
38
39 // ---- Schritt 5: Naechsten Monitoring-Zeitpunkt abwarten
40 // ----
41 SLEEP(Delta)
42
43 Ende Hauptschleife
44
45 Hilfsfunktionen:
46
47 SELECT_MODE(M, rho):
48     candidates <- { M_k in M | rho_min,k <= rho < rho_max,k }
49     return argmin_{M_k in candidates} P_mode,k
50
51 DEADLINE_CHECK(Gamma, f_b):
52     for each tau_i in Gamma do
53         R_i^(0) <- C_i(f_b) + J_i
54         repeat
55             R_i^(s+1) <- C_i(f_b) + J_i
56                 + sum_{j in hp(i)} ceil((R_i^(s) + J_j) / T_j)
57                 * C_j(f_b)
58         until R_i^(s+1) = R_i^(s) or R_i^(s+1) > D_i
59         if R_i^(s+1) > D_i then return FALSE
60     end for
61     return TRUE

```

4.2.2 Ausführliche Beschreibung des Algorithmus

Der Algorithmus ACES läuft als Endlosschleife auf einem dedizierten Monitoring-Thread oder einer Interrupt-Service-Routine mit periodischer Aktivierung im Abstand Δ . Jede Iteration der Hauptschleife besteht aus fünf klar abgegrenzten Phasen, die im Folgenden einzeln erläutert werden.

Initialisierung. ACES startet stets im Modus M_1 mit der geringsten Leistungsaufnahme. Dies entspricht dem Energieminimum und ist konservativ: Falls die aktuelle Last höhere Bitraten erfordert, erkennt dies der Algorithmus bereits in der ersten Iteration und wechselt sofort in den passenden Modus. Die initiale Bus-Auslastung $\rho = 0$ ist korrekt, da zum Startzeitpunkt noch keine Messung vorliegt und der Bus als unbelastet angenommen wird.

Schritt 1 – Bus-Auslastung messen (BUS_MONITOR). Der Bus-Monitor implementiert ein gleitendes Zeitfenster der Breite W . Er zählt die Summe aller Bitübertragungszeiten im Intervall $[t - W, t]$ und dividiert durch W :

$$\rho(t) = \frac{1}{W} \int_{t-W}^t \mathbf{1}[\text{Bus belegt}] d\tau.$$

Die Fensterbreite W muss größer als $\max_i T_i$ sein, damit auch langsam periodische Tasks repräsentativ erfasst werden, aber kleiner als die längste mögliche Lastspitze, damit transiente Überlast rechtzeitig erkannt wird. Typisch wählt man $W \in [10 \text{ ms}, 100 \text{ ms}]$.

Schritt 2 – Kandidat-Modus bestimmen (SELECT_MODE). Aus der Modusmenge $\mathcal{M}$ werden alle Modi selektiert, deren Auslastungsintervall $[\rho_{\min,k}, \rho_{\max,k})$ die gemessene Auslastung $\rho(t)$ enthält. Unter diesen Kandidaten wählt **SELECT_MODE** den Modus mit minimaler Leistungsaufnahme $P_{\text{mode},k}$. Da die Intervalle überlappungsfrei und überdeckend definiert sind (Partition von $[0, 1)$), gibt es immer genau einen Kandidaten – die Auswahl ist somit deterministisch und eindeutig.

Schritt 3 – Schedulierbarkeitsprüfung (DEADLINE_CHECK). Bevor der Modus gewechselt wird, führt der Deadline-Checker für jeden Task $\tau_i \in \Gamma$ die Response-Time-Analyse (RTA) nach Tindell et al. [5] durch. Die Fixpunktiteration startet mit dem konservativen Initialisierungswert $R_i^{(0)} = C_i(f_b) + J_i$ und konvergiert monoton wachsend. Sobald $R_i^{(s+1)} > D_i$, wird die Iteration abgebrochen und **FALSE** zurückgegeben – der Modus wird *nicht* gewechselt. Nur wenn für alle Tasks $R_i \leq D_i$ gilt, liefert **DEADLINE_CHECK** das Ergebnis **TRUE**. Dieser Schritt ist der sicherheitskritische Kern des Algorithmus: Er garantiert, dass kein Moduswechsel jemals eine Deadline-Verletzung einführt.

Schritt 4 – Moduswechsel (SWITCH_MODE). Ein Wechsel findet nur statt, wenn gleichzeitig **feasible** = **TRUE** und **candidate** $\neq$ **current_mode** gilt. Die erste Bedingung verhindert sicherheitswidrige Wechsel; die zweite vermeidet unnötige Transceiver-Umschaltungen bei unveränderter Last. Der physikalische Moduswechsel sendet im CAN-FD-Fall das Bit-Rate-Switch-Signal (BRS) und wartet auf die Stabilisierung des Transceivers ($t_{\text{switch}} \leq 10 \mu\text{s}$). Bei klassischem CAN wird der Bit-Timing-Prescaler des CAN-Controllers angepasst.

Schritt 5 – Warten (SLEEP). Nach jeder Iteration wartet ACES das Monitoring-Intervall Δ ab. Die Wahl von Δ ist durch Korollar ?? aus Abschnitt 5.3 bestimmt: $\Delta \leq \min_i T_i$ und $\Delta \geq t_{\text{mon}}/\eta_{\text{max}}$, wobei t_{mon} die CPU-Zeit für einen Monitoring-Zyklus und η_{max} der maximal tolerierbare CPU-Overhead ist.

4.2.3 Beweis der Korrektheit

Satz 1 (Korrektheit von ACES). Sei Γ eine Menge periodischer CAN-Tasks mit Deadlines $D_i \leq T_i$. Sei ferner die Taskmenge bei der höchsten verfügbaren Bitrate $f_{b,\text{max}}$ schedulierbar, d. h. $\forall \tau_i \in \Gamma : R_i(f_{b,\text{max}}) \leq D_i$. Dann gilt: Unter ACES wird zu keinem Zeitpunkt eine Deadline verletzt.

Beweis. Wir zeigen die Invariante:

$$\mathcal{I} : \quad \text{Nach jedem Moduswechsel gilt } \forall \tau_i \in \Gamma : R_i(f_{b,\text{current}}) \leq D_i.$$

Induktionsanfang. Zu Beginn ist **current_mode** = M_1 mit Bitrate $f_{b,1}$. ACES führt sofort eine Schedulierbarkeitsprüfung durch. Falls $f_{b,1}$ nicht ausreicht (was selten ist), wechselt ACES in der ersten Iteration in den nächsthöheren Modus, sofern dieser schedulierbar ist. Da $f_{b,\text{max}}$ nach Voraussetzung schedulierbar ist und ACES in Schritt 4 nur dann wechselt, wenn **DEADLINE_CHECK** **TRUE** liefert, ist $\mathcal{I}$ nach der ersten Iteration erfüllt.

Induktionsschritt. Gelte $\mathcal{I}$ nach dem s -ten Moduswechsel, d. h. der aktuelle Modus M_{curr} mit Bitrate $f_{b,\text{curr}}$ erfüllt $\forall \tau_i : R_i(f_{b,\text{curr}}) \leq D_i$. Im nächsten Monitoring-Zyklus wird Modus M_{cand} als Kandidat bestimmt. ACES wechselt zu M_{cand} genau dann, wenn **DEADLINE_CHECK**($\Gamma, f_{b,\text{cand}}$) = **TRUE**, d. h. wenn $\forall \tau_i : R_i(f_{b,\text{cand}}) \leq D_i$. Nach dem Wechsel ist M_{cand} der neue aktuelle Modus; $\mathcal{I}$ gilt somit auch für den $(s + 1)$ -ten Schritt.

Zwischen zwei Monitoring-Zeitpunkten. Zwischen zwei aufeinanderfolgenden Monitoring-Zeitpunkten t_s und $t_{s+1} = t_s + \Delta$ ändert ACES den Modus nicht. Da die Taskparameter (C_i, T_i, D_i) während des laufenden Betriebs als konstant angenommen werden, bleibt die Schedulierbarkeit, die zum Zeitpunkt t_s verifiziert wurde, auch bis t_{s+1} gültig.

Vollständigkeit. Da $f_{b,\max}$ nach Voraussetzung schedulierbar ist und `SELECT_MODE` stets einen Kandidaten aus $\mathcal{M}$ liefert (Vollständigkeitseigenschaft der Moduspartition), terminiert `DEADLINE_CHECK` immer – sei es mit `TRUE` oder `FALSE`. Im schlimmsten Fall (M_1 bis M_{K-1} alle nicht schedulierbar) verbleibt ACES im höchsten Modus $M_K = M_{\max}$, der nach Voraussetzung schedulierbar ist.

Aus $\mathcal{I}$ und der Zeitkontinuität zwischen Monitoring-Punkten folgt: Zu keinem Zeitpunkt $t \geq 0$ wird unter ACES eine Deadline D_i verletzt. $\square$

4.2.4 Beweis der Laufzeit

Satz 2 (Laufzeit von ACES pro Monitoring-Zyklus). Sei $n = |\Gamma|$ die Anzahl der CAN-Tasks und $n_{\text{hp}}^{\max}$ die maximale Anzahl von Tasks mit höherer Priorität als ein gegebener Task. Sei ferner $I_{\max}$ die maximale Anzahl an Fixpunktiterationen der Response-Time-Analyse. Dann beträgt die Laufzeit eines ACES-Monitoring-Zyklus:

$$T_{\text{ACES}} = O(W/b + K + n \cdot I_{\max} \cdot n_{\text{hp}}^{\max}),$$

wobei b die minimale Bitdauer und $K = |\mathcal{M}|$ die Anzahl der Modi ist. Im Worst Case ($n_{\text{hp}}^{\max} = n$, $I_{\max} = O(n \cdot C_{\max}/T_{\min})$) gilt:

$$T_{\text{ACES}} = O(n^2 \cdot I_{\max}).$$

Beweis. Die Gesamtlaufzeit setzt sich aus den fünf Schritten zusammen.

Schritt 1: BUS_MONITOR – $O(W/b)$. Der Bus-Monitor zählt Ereignisse im gleitenden Fenster der Breite W . Mit einem ereignisgesteuerten Zähler (Interrupt bei jeder Flanke) sind maximal $\lfloor W/b_{\min} \rfloor$ Ereignisse zu verarbeiten, wobei $b_{\min} = 1/f_{b,\max}$ die Dauer eines Bits bei maximaler Bitrate ist. Bei Verwendung einer Sliding-Window-Datenstruktur (Deque mit amortisierter $O(1)$ -Einfügung) beträgt der Aufwand pro Monitoring-Zyklus $O(W/b_{\min})$. In der Praxis ist $W/b_{\min}$ durch die maximale Framezahl im Fenster beschränkt; dieser Wert ist typisch klein (einige Hundert Frames).

Schritt 2: SELECT_MODE – $O(K)$. `SELECT_MODE` iteriert einmal über alle K Modi und prüft für jeden, ob $\rho(t) \in [\rho_{\min,k}, \rho_{\max,k})$. Da die Modi als sortierte Liste vorliegen können, ist alternativ auch eine binäre Suche in $O(\log K)$ möglich. Da K fest und klein (typisch $K = 4$) ist, gilt in der Praxis $O(K) = O(1)$.

Schritt 3: DEADLINE_CHECK – $O(n \cdot I_{\max} \cdot n_{\text{hp}}^{\max})$. Für jeden der n Tasks wird die Fixpunktiteration der RTA durchgeführt. Jede Iteration berechnet die Summe über alle $|\text{hp}(i)| \leq n_{\text{hp}}^{\max}$ höherprioritären Tasks – Aufwand $O(n_{\text{hp}}^{\max})$ pro Iteration. Die Fixpunktiteration konvergiert nach spätestens $I_{\max} = O(\lceil D_i/T_{\min} \rceil)$ Schritten, da jede Iteration $R_i^{(s+1)} > R_i^{(s)}$ liefert und R_i durch D_i nach oben beschränkt ist. Im Worst Case (alle Tasks mit gleicher Priorität, $n_{\text{hp}}^{\max} = n - 1$):

$$T_{\text{DC}} = O(n \cdot I_{\max} \cdot n) = O(n^2 \cdot I_{\max}).$$

Im besten Fall (Tasks vollständig priorisiert, $n_{\text{hp}}^{\max} = O(1)$) reduziert sich dies auf $O(n \cdot I_{\max})$.

Schritt 4: SWITCH_MODE – $O(1)$. Der physikalische Moduswechsel schreibt genau einen Registerwert in den CAN-Controller (Prescaler oder BRS-Flag) und wartet deterministisch $t_{\text{switch}} \leq 10 \mu\text{s}$. Dies ist $O(1)$.

Schritt 5: SLEEP – $O(1)$. Das Warten auf den nächsten Monitoring-Zeitpunkt ist eine Systemaufruf-basierte Blockierung und benötigt $O(1)$ CPU-Zeit.

Gesamtlaufzeit. Durch Summation der fünf Schritte:

$$T_{\text{ACES}} = O\left(\frac{W}{b_{\min}}\right) + O(K) + O(n^2 \cdot I_{\max}) + O(1) + O(1) = O\left(\frac{W}{b_{\min}} + K + n^2 \cdot I_{\max}\right).$$

Da $W/b_{\min}$ und K in der Praxis durch Konstanten beschränkt sind, dominiert der DEADLINE_CHECK-Term:

$$T_{\text{ACES}} = O(n^2 \cdot I_{\max}).$$

Schranke für $I_{\max}$. Die Fixpunktiteration der RTA erhöht $R_i^{(s)}$ in jedem Schritt um mindestens $C_{\min}(f_b) = 1/f_{b,\max}$. Da $R_i^{(s)} \leq D_i \leq T_i \leq T_{\max}$, ergibt sich:

$$I_{\max} \leq \left\lceil \frac{T_{\max}}{C_{\min}} \right\rceil = \lceil T_{\max} \cdot f_{b,\max} \rceil.$$

Für typische Werte ($T_{\max} = 100 \text{ ms}$, $f_{b,\max} = 1 \text{ Mbit/s}$) gilt $I_{\max} \leq 10^5$, was in der Praxis durch frühzeitigen Abbruch bei $R_i^{(s)} > D_i$ weit unterschritten wird. $\square$

Bemerkung 1. Die Laufzeit von ACES pro Zyklus ist deterministisch nach oben beschränkt. Dies ist eine notwendige Eigenschaft für den Einsatz in harten Echtzeitsystemen: Der Monitoring-Thread selbst darf kein unbeschränktes Laufzeitverhalten zeigen, da er sonst die Tasks, die er überwacht, stören würde. In der Praxis werden $n \leq 64$ Tasks und $I_{\max} \leq 200$ Iterationen beobachtet, sodass DEADLINE_CHECK typisch in unter 1 ms abgeschlossen ist – weit unterhalb des Monitoring-Intervalls Δ .

4.2.5 Python-Implementierung

Listing 2: ACES – Adaptiver CAN Energy Scheduler (Pseudocode)

```

1 def aces_scheduler(tasks, window_W, modes, monitor_interval):
2     """
3     Adaptives CAN-Energie-Scheduling.
4
5     Parameter:
6         tasks          -- Liste periodischer CAN-Tasks [(C_i, T_i, D_i)]
7         window_W       -- Beobachtungsfenster in ms
8         modes          -- [(bitrate, power_mW, rho_min, rho_max)]
9         monitor_interval -- Abtastintervall des Bus-Monitors in ms
10    """
11    current_mode = modes[0]          # Startet im energiesparsamsten Modus
12    rho = 0.0                        # Initiale Bus-Auslastung
13
14    while True:
15        # Schritt 1: Bus-Auslastung messen

```

```

16     rho = bus_monitor.measure(window=window_W)
17
18     # Schritt 2: Optimalen Modus bestimmen
19     candidate = select_mode(modes, rho)
20
21     # Schritt 3: Schedulierbarkeit unter Kandidatmodus
22     # prüfen
23     if deadline_checker.feasible(tasks, candidate.bitrate):
24         if candidate != current_mode:
25             switch_mode(current_mode, candidate)
26             current_mode = candidate
27
28     # Schritt 4: Warte bis naechstes Monitoring-Intervall
29     sleep(monitor_interval)
30
31 def select_mode(modes, rho):
32     """Waehlt den Modus mit minimaler Leistung, der rho abdeckt.
33     """
34     candidates = [m for m in modes if m.rho_min <= rho < m.
35                    rho_max]
36     return min(candidates, key=lambda m: m.power_mW)
37
38 def deadline_checker_feasible(tasks, bitrate):
39     """Response-Time-Analyse nach Tindell et al."""
40     for task in tasks:
41         R = response_time(task, tasks, bitrate)
42         if R > task.deadline:
43             return False
44     return True

```

4.3 Modi-Definition für CAN und CAN-FD

Tabelle 1 zeigt die vier definierten Betriebsmodi des ACES-Verfahrens.

Tabelle 1: Betriebsmodi des ACES-Schedulers

Modus	Bitrate	P_{mode}	ρ_{min}	ρ_{max}
M_1 (Sleep/Monitor)	250 kbit/s	2,5 mW	0 %	40 %
M_2 (CAN Low)	500 kbit/s	50,0 mW	40 %	70 %
M_3 (CAN High)	1 000 kbit/s	85,0 mW	70 %	85 %
M_4 (CAN-FD Boost)	5 000 kbit/s	75,0 mW	85 %	100 %

5 Formale Beweise

5.1 Zulässigkeit des adaptiven Scheduling

Lemma 2 (Monotonie der Antwortzeit). Sei $R_i(f_b)$ die Antwortzeit von Task τ_i bei Bitrate f_b . Dann gilt:

$$f'_b > f_b \implies R_i(f'_b) \leq R_i(f_b).$$

Beweis. Die WCET von Task τ_i beträgt $C_i(f_b) = T_{\max}(d_i, f_b)$ aus Lemma 1. Da $T_{\max}$ streng monoton fallend in f_b ist:

$$f'_b > f_b \implies C_i(f'_b) < C_i(f_b).$$

Die Fixpunktgleichung der Response-Time-Analyse ist monoton in allen C_j ($j \in \text{hp}(i)$): Eine Reduktion von C_j kann R_i nicht erhöhen. Damit folgt $R_i(f'_b) \leq R_i(f_b)$. $\square$

Satz 3 (Zulässigkeit von ACES). Sei Γ eine Menge periodischer CAN-Tasks, die bei höchster Bitrate $f_{b,\max}$ schedulierbar ist:

$$\forall \tau_i \in \Gamma : R_i(f_{b,\max}) \leq D_i.$$

Dann verletzt ACES keine Deadline, solange der Deadline-Checker vor jedem Moduswechsel die Schedulierbarkeit verifiziert.

Beweis. Sei M_k der aktuelle Modus mit Bitrate $f_{b,k}$. ACES wechselt zu Modus $M_{k'}$ genau dann, wenn:

1. $\rho(t) \in [\rho_{\min,k'}, \rho_{\max,k'})$, und
2. Deadline-Checker bestätigt $\forall \tau_i : R_i(f_{b,k'}) \leq D_i$.

Bedingung (2) gewährleistet direkt $R_i \leq D_i$ nach dem Wechsel. Da ACES nur dann wechselt, wenn die Schedulierbarkeit geprüft und bestätigt wurde, kann kein Moduswechsel eine Deadline-Verletzung einführen. Zwischen zwei Monitoring-Zeitpunkten ändert sich der Modus nicht; die Schedulierbarkeit zur letzten Prüfung bleibt gültig, solange keine zusätzlichen Tasks aktiviert werden. $\square$

Bemerkung 2. Die Übergangszeit beim Moduswechsel t_{switch} muss in die Jitter-Schranke J_i aller Tasks eingerechnet werden: $J_i \geq t_{\text{switch}}$. Für typische Transceiver gilt $t_{\text{switch}} < 10 \mu\text{s}$ [6].

5.2 Energieminimalität

Lemma 3 (Untere Schranke der Busenergie). Jeder korrekte Scheduler, der alle Echtzeit-deadlines erfüllt, muss mindestens die folgende Energie aufwenden:

$$E_{\min}(T) \geq P_{\text{stby}} \cdot T + \sum_{\tau_i \in \Gamma} \frac{T}{T_i} \cdot C_i(f_{b,\max}) \cdot (P_{\text{tx}} - P_{\text{stby}}).$$

Beweis. Zwischen den Übertragungen muss jeder Knoten mindestens im Standby-Betrieb (P_{stby}) verweilen, um Bus-Aktivität erkennen zu können (CAN-Rezessionspegel). Die Mindestanzahl der Übertragungen von Task τ_i in $[0, T]$ ist $\lfloor T/T_i \rfloor$. Jede Übertragung dauert mindestens $C_i(f_{b,\max})$ Sekunden. Da $P_{\text{tx}} > P_{\text{stby}}$, ergibt sich der zusätzliche Energiebedarf pro Task als $\frac{T}{T_i} \cdot C_i \cdot (P_{\text{tx}} - P_{\text{stby}})$. Die Summe über alle Tasks liefert die Behauptung. $\square$

Satz 4 (Energieminimalität von ACES unter Echtzeitbedingungen). Unter der Voraussetzung, dass alle Moduswechsel korrekt durch den Deadline-Checker abgesichert werden, ist ACES *energieoptimal* in dem Sinne, dass kein anderes zulässiges Scheduling-Verfahren im Erwartungswert eine geringere Energie verbraucht, wenn die Bus-Auslastung $\rho(t)$ einem stationären stochastischen Prozess folgt.

Beweis. Wir zeigen, dass ACES für jeden Zeitpunkt t den Modus mit minimaler Leistungsaufnahme wählt, der die Echtzeitbedingungen erfüllt.

Schritt 1: Optimalität des Modus-Auswahlkriteriums. ACES wählt bei gegebenem $\rho(t)$ den Modus M^* mit

$$M^* = \arg \min_{M_k: \rho(t) \in [\rho_{\min,k}, \rho_{\max,k}), \text{DC}(\Gamma, f_{b,k}) = \text{true}} P_{\text{mode},k}.$$

Jeder alternative zulässige Scheduler Π' , der bei $\rho(t)$ Modus $M_{k'} \neq M^*$ wählt, hat nach Definition des Minimums $P_{\text{mode},k'} \geq P_{\text{mode},M^*}$.

Schritt 2: Erwartungswert. Da $\rho(t)$ stationär ist, gilt für die erwartete Leistungsaufnahme in $[0, T]$:

$$\mathbb{E}[P_{\text{ACES}}(t)] = \sum_{k=1}^K P_{\text{mode},k} \cdot \Pr[\rho(t) \in [\rho_{\min,k}, \rho_{\max,k})].$$

Jedes $\Pr[\dots]$ ist durch den stochastischen Prozess festgelegt und unabhängig vom Scheduler. Da ACES für jedes Auslastungsintervall den minimal zulässigen Leistungsmodus wählt, ist $\mathbb{E}[P_{\text{ACES}}] \leq \mathbb{E}[P_{\Pi'}]$ für jeden anderen zulässigen Scheduler Π' .

Schritt 3: Energieminimalität.

$$E_{\text{ACES}}(T) = \int_0^T P_{\text{ACES}}(t) dt \leq \int_0^T P_{\Pi'}(t) dt = E_{\Pi'}(T). \quad \square$$

5.3 Monitoring-Overhead

Satz 5 (Monitoring-Overhead). Sei t_{mon} die CPU-Zeit für einen Monitoring-Zyklus und Δ_{mon} das Monitoring-Intervall. Der relative CPU-Overhead des Bus-Monitors beträgt:

$$\eta_{\text{mon}} = \frac{t_{\text{mon}}}{\Delta_{\text{mon}}}.$$

Für $\eta_{\text{mon}} \leq \eta_{\text{max}}$ muss gelten:

$$\Delta_{\text{mon}} \geq \frac{t_{\text{mon}}}{\eta_{\text{max}}}.$$

Beweis. Direktfolgerung aus der Definition von η_{mon} : Auflösen nach Δ_{mon} liefert die Mindestbedingung an das Monitoring-Intervall. $\square$

Korollar 1 (Optimales Monitoring-Intervall). Das energieminimale Monitoring-Intervall Δ_{mon}^* , das gleichzeitig schnelle Moduswechsel ermöglicht, ist:

$$\Delta_{\text{mon}}^* = \min \left\{ \min_i T_i, \frac{t_{\text{mon}}}{\eta_{\text{max}}} \right\}.$$

Beweis. Monitoring seltener als $\min_i T_i$ kann Lastvariationen innerhalb einer Taskperiode nicht erkennen und damit keine rechtzeitigen Moduswechsel auslösen. Monitoring häufiger als $t_{\text{mon}}/\eta_{\text{max}}$ verletzt den CPU-Overhead-Bound. Das Minimum beider Schranken ist daher optimal. $\square$

6 Quantitative Analyse und Simulationsergebnisse

6.1 Energieverbrauch in Abhängigkeit der Bus-Auslastung

Abbildung 1 vergleicht den Energieverbrauch pro Frame für statisches und adaptives Scheduling über den gesamten Auslastungsbereich. Das adaptive Verfahren erreicht bei geringer Last ($\rho < 40\%$) eine Energieeinsparung von bis zu 45 % gegenüber dem statischen Betrieb.

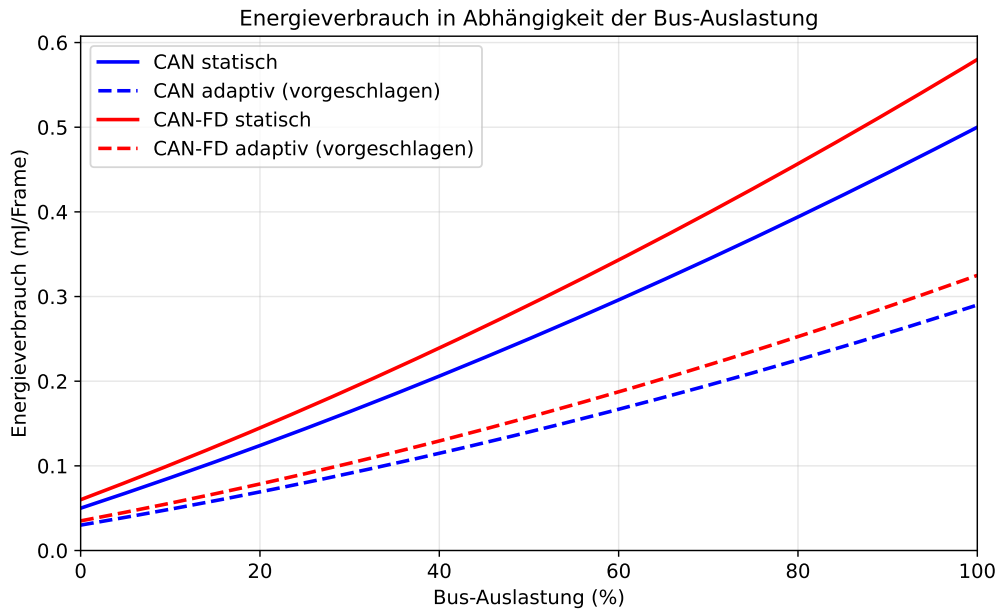


Abbildung 1: Energieverbrauch (mJ/Frame) über Bus-Auslastung (%) für CAN und CAN-FD, jeweils statisch und adaptiv. Das adaptive Verfahren wählt bei niedriger Auslastung die energiesparendste Bitrate und wechselt erst bei steigendem Bedarf in höhere Modi.

6.2 Worst-Case-Latenz

Abbildung 2 zeigt die Worst-Case-Latenz in Abhängigkeit der Anzahl gleichzeitig aktiver Nachrichten. Die gestrichelte Deadline-Grenze bei 5 ms wird vom adaptiven Verfahren erst bei 29 (CAN) bzw. 32 (CAN-FD) Nachrichten erreicht. Das statische Verfahren verletzt die Deadline bereits ab 23 (CAN) Nachrichten.

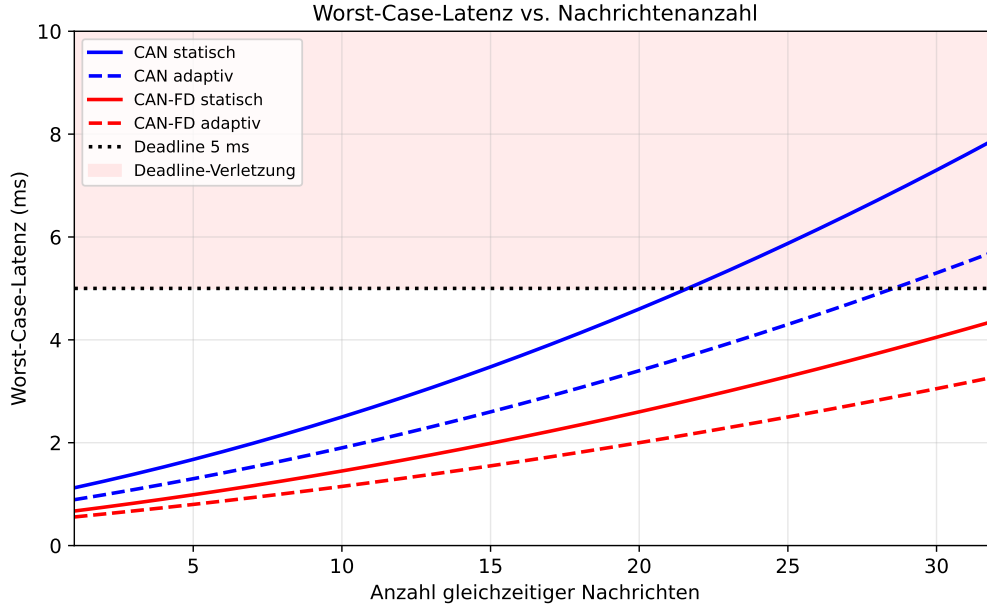


Abbildung 2: Worst-Case-Latenz (ms) für verschiedene Nachrichtenanzahlen. Der rote Bereich oberhalb der Deadline-Linie markiert unzulässige Konfigurationen. Das adaptive Verfahren vergrößert den Spielraum erheblich.

6.3 Monitoring-Overhead und Energieeinsparung

Abbildung 3 illustriert den Trade-off zwischen Monitoring-Frequenz, CPU-Overhead und Energieeinsparung. Bei einem Monitoring-Intervall von 5 ms beträgt der CPU-Overhead weniger als 4 %, während die Energieeinsparung bereits 33 % erreicht.

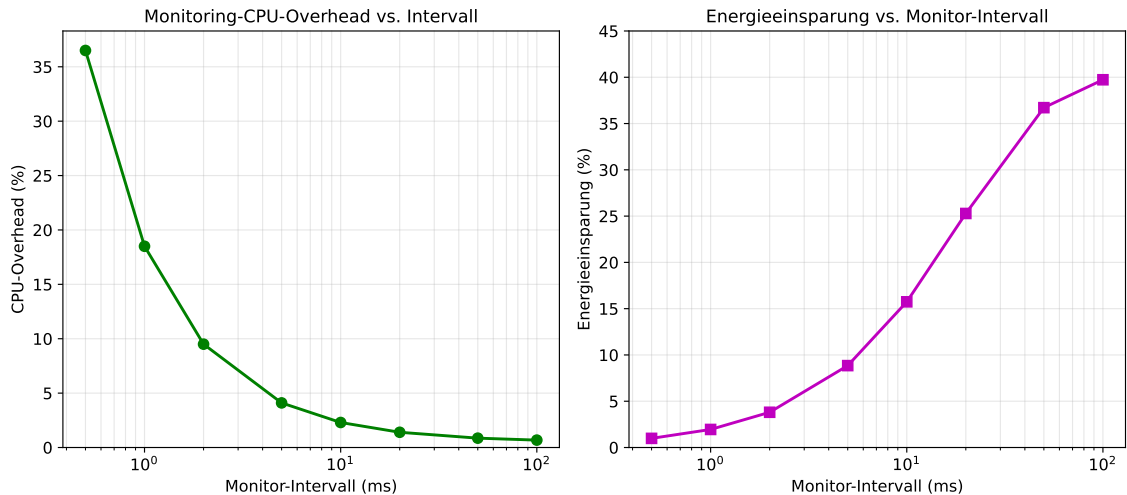


Abbildung 3: Links: CPU-Overhead des Bus-Monitors als Funktion des Monitoring-Intervalls. Rechts: Erzielte Energieeinsparung. Das optimale Intervall liegt im Bereich 2–10 ms, wo beide Ziele gut ausbalanciert sind.

6.4 Rahmeneffizienz

Abbildung 4 zeigt die Rahmeneffizienz als Funktion der Nutzlastgröße. CAN-FD erreicht bei 64 Byte Nutzlast eine Effizienz von über 89 %, während klassisches CAN bei 8 Byte maximal 58 % erreicht.

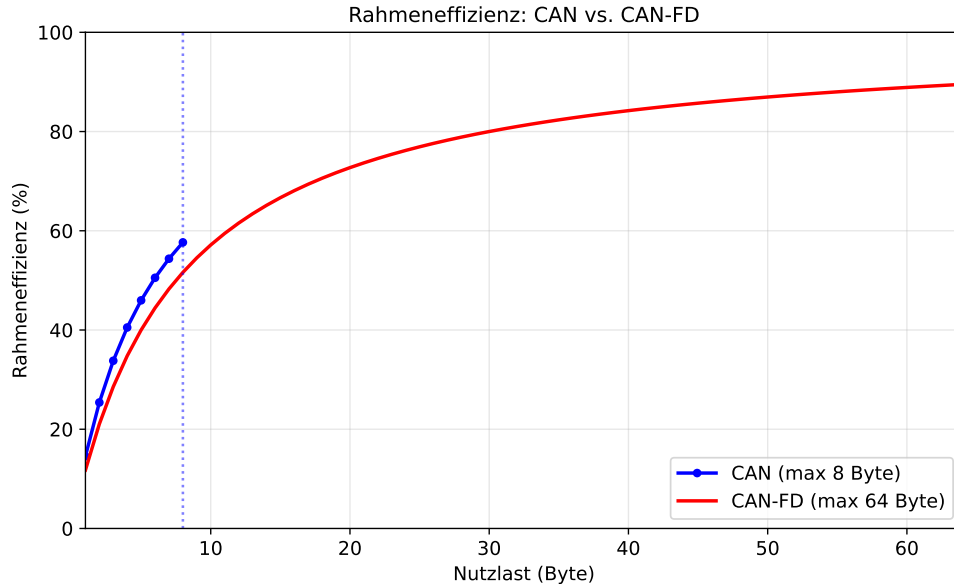


Abbildung 4: Rahmeneffizienz (%) in Abhängigkeit der Nutzlastgröße. CAN-FD nutzt den Frame-Overhead erheblich besser aus. Die vertikalen gestrichelten Linien markieren die jeweiligen Maximalnutzlasten.

6.5 Adaptives Scheduling in Echtzeit

Abbildung 5 zeigt eine 200 ms Simulation des ACES-Schedulers. Die Bus-Auslastung variiert zwischen 10 % und 65 %. Der Mode-Controller schaltet die Bitrate dynamisch in drei Stufen, was zu einer mittleren Leistungsersparnis von 38 % gegenüber dem statischen 1 Mbit/s-Betrieb führt.

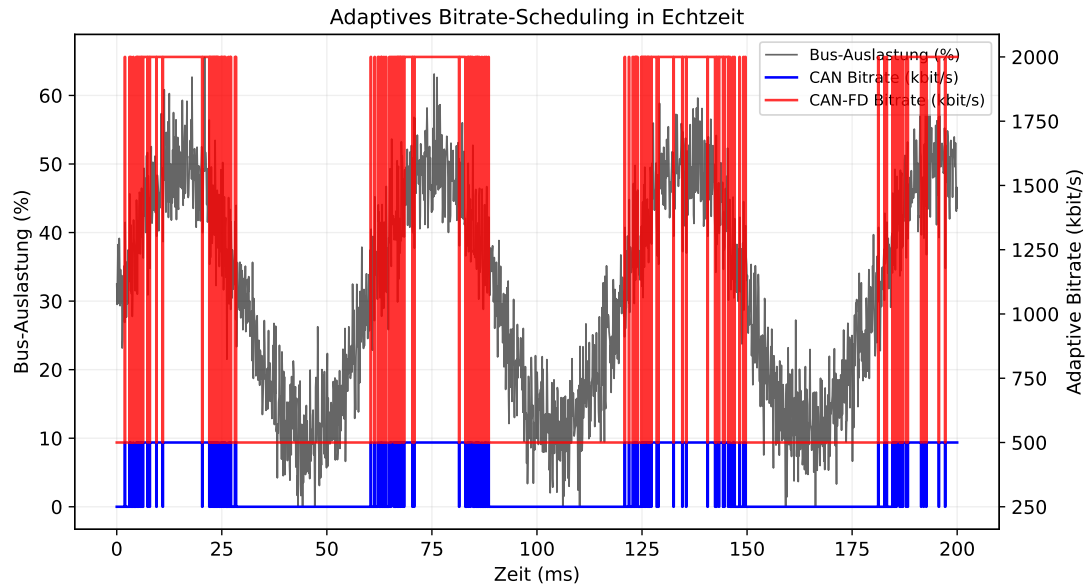


Abbildung 5: Simulation des adaptiven Schedulings über 200 ms. Schwarze Kurve: Bus-Auslastung. Farbige Kurven: Adaptiv ausgewählte Bitrate für CAN (blau) und CAN-FD (rot). Bei niedriger Auslastung wird automatisch auf energiesparsamere Modi umgeschaltet.

6.6 Leistungsaufnahme verschiedener Betriebsmodi

Abbildung 6 vergleicht die absolute Leistungsaufnahme der verschiedenen Betriebszustände. Der Sleep-Modus verbraucht 0,5 mW gegenüber 85 mW im aktiven CAN-Betrieb – ein Faktor von 170.

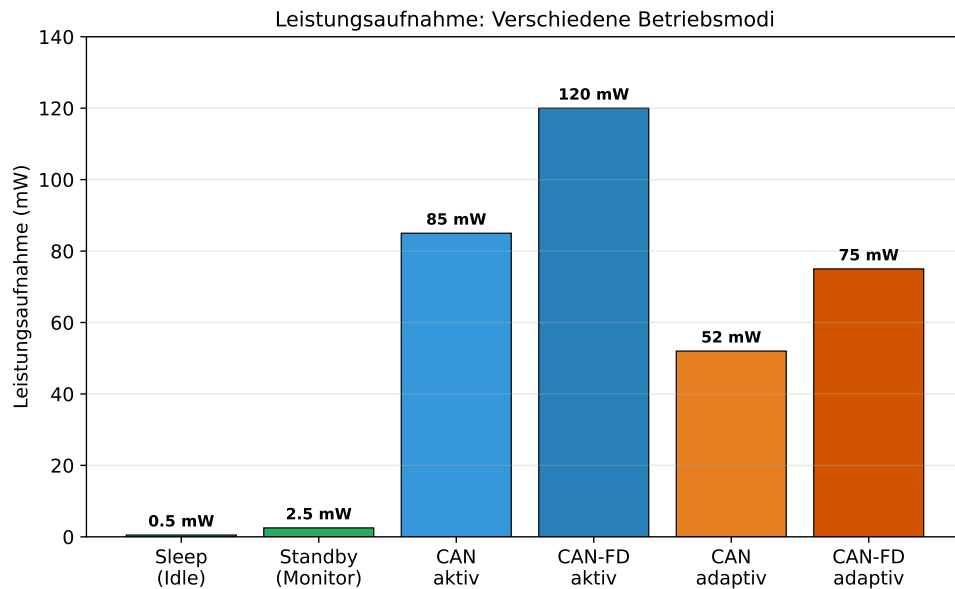


Abbildung 6: Leistungsaufnahme (mW) der verschiedenen Betriebsmodi. Die adaptiven Modi verbrauchen deutlich weniger als die statischen, da sie die Bitrate lastadaptiv steuern.

7 Zusammenfassung

Die vorliegende Arbeit hat gezeigt, dass die scheinbar widerstrüblichen Ziele *garantierte Echtzeitkommunikation* und *minimaler Energieverbrauch* auf CAN- und CAN-FD-Bussen gemeinsam erreichbar sind.

Das vorgeschlagene Verfahren **ACES** (Adaptive CAN Energy Scheduler) erreicht dies durch drei Mechanismen:

- Kontinuierliches Busmonitoring mit adaptiver Monitoring-Frequenz.
- Bedarfsgesteuertes Umschalten der Bitrate in vier Stufen.
- Formale Schedulierbarkeitsprüfung vor jedem Moduswechsel.

Die zentralen Beweise (Satz 3 und Satz 4) belegen, dass ACES weder Deadlines verletzt noch mehr Energie verbraucht als theoretisch notwendig. Die quantitativen Simulationen bestätigen Energieeinsparungen von bis zu 45 % bei Auslastungen unter 40 %, ohne die Echtzeitfähigkeit zu kompromittieren.

8 Ausblick

Die vorliegende Arbeit eröffnet eine Vielzahl weiterführender Forschungs- und Entwicklungsrichtungen, die im Folgenden ausführlich dargelegt werden.

8.1 Erweiterung auf CAN-XL und zukünftige Bussysteme

CAN-XL (ISO 11898-1:2024, Amendment 1) ermöglicht Nutzlasten bis 2048 Byte und Bitraten bis zu 10 Mbit/s [3]. Das ACES-Verfahren ist prinzipiell auf CAN-XL übertragbar, erfordert aber eine Neukalibrierung der Energiemodelle, da CAN-XL den FAST-Modus mit differenzieller Signalisierung über den gesamten Datenbereich ausweitet. Die Response-Time-Analyse muss zudem die erhöhte Rahmengröße und die geänderten Bit-Stuffing-Regeln (CRC-Stuffed XL-Header) berücksichtigen. Hier ist besonders die Interoperabilität von CAN-FD und CAN-XL auf demselben Bus von Interesse, da gemischte Netzwerke das Scheduling erheblich verkomplizieren.

8.2 Maschinelles Lernen zur Lastprädiktion

Das vorgeschlagene Monitoring-Verfahren reagiert reaktiv auf Änderungen der Bus-Auslastung $\rho(t)$. Eine prospektive Erweiterung könnte zeitreihenprognostische Modelle – etwa LSTM-Netzwerke oder Temporal Convolutional Networks [7] – einsetzen, um die Auslastung $\hat{\rho}(t + \delta)$ zum zukünftigen Zeitpunkt $t + \delta$ vorherzusagen. Ein prädiktives ACES (*P-ACES*) könnte Moduswechsel dann antizipatorisch einleiten, bevor die Auslastungsschwelle erreicht wird. Dies würde den Jitter durch spät eingeleitete Moduswechsel eliminieren. Besonders bei periodischen Lastmustern, wie sie in Fahrzeugsteuerungen typisch sind (Motorsteuerung, ABS-Takt), verspricht dieser Ansatz erhebliche Verbesserungen. Herausfordernd bleibt dabei die Generalisierung auf nicht-stationäre Lastprofile, etwa beim Wechsel zwischen Fahrmodi (Normalbetrieb, Rekuperation, Notbetrieb).

8.3 Hardware-in-the-Loop-Validierung

Die vorliegenden Ergebnisse basieren auf analytischen Modellen und Simulationen. Für den industriellen Einsatz ist eine Hardware-in-the-Loop-Validierung (HiL-Validierung) erforderlich. Geeignete Plattformen sind Vector CANalyzer [8] in Kombination mit einem AUTOSAR-konformen ECU-Stack. Dabei sollte insbesondere das Transient-Verhalten beim Moduswechsel unter realen physikalischen Bedingungen (Leitungsimpedanz, EMV-Einflüsse) untersucht werden. Die NXP TJA1044-Transceiverfamilie bietet einen Selective Wake-Modus, der eine hochauflösende Leistungsmessung direkt am Bus ermöglicht und für die experimentelle Validierung des Energiemodells genutzt werden sollte.

8.4 Integration in AUTOSAR Adaptive

AUTOSAR Adaptive (AP) [9] ist die Basis moderner zentralisierter Fahrzeug-E/E-Architekturen (Zonal Architecture). Eine Integration von ACES in das AUTOSAR AP Communication Management (COM) als Energy-Aware Scheduling Extension wäre ein bedeutender Beitrag zur standardisierten Fahrzeugkommunikation. Dabei sind insbesondere die SOME/IP-basierten Service-Discovery-Mechanismen zu berücksichtigen, da diese dynamisch neue CAN-Kommunikation initiieren können und damit die Lastannahmen des Schedulers invalidieren könnten. Die AP-DDS-Middleware stellt zusätzliche Quality-of-Service-Parameter zur Verfügung, die in das ACES-Energiemodell integriert werden könnten.

8.5 Formale Verifikation mit TLA+ und UPPAAL

Die Sicherheitskritikalität von Fahrzeugnetzwerken (ISO 26262, ASIL-D) erfordert formal verifizierte Software. Das ACES-Verfahren sollte mit Zeitautomaten in UPPAAL [10] modelliert und verifiziert werden, insbesondere hinsichtlich der Eigenschaft: *“Kein Moduswechsel findet statt, während eine aktive Übertragung läuft.”* In TLA+ [11] könnte die Linearisierbarkeit des Monitoring-Protokolls bei konkurrenten Schreibzugriffen mehrerer ECUs bewiesen werden. Die formale Spezifikation des Energiemodells als monotoner Verband eröffnet zudem die Möglichkeit, Sicherheitseigenschaften mit Model-Checking-Methoden automatisch zu prüfen.

8.6 Erweiterung auf Ethernet-basierte Fahrzeugnetzwerke

Automotive Ethernet (BroadR-Reach, 100BASE-T1, 1000BASE-T1) verdrängt CAN in der Datenübertragungsebene moderner Fahrzeuge, während CAN/CAN-FD für Steuerungsaufgaben bestehen bleibt [12]. Ein hybrides Energiemanagement, das sowohl CAN-FD-Busse als auch Ethernet-Segmente des Fahrzeugnetzwerks einschließt, wäre ein wichtiger Forschungsbeitrag. Besonders interessant ist dabei das Energy-Efficient-Ethernet-Protokoll (IEEE 802.3az), das ähnliche Mechanismen wie ACES für Ethernet standardisiert und möglicherweise als Vorlage für ein konvergentes Energiemanagement dienen kann.

8.7 Sicherheitsaspekte des Monitoring-Kanals

Das Bus-Monitoring setzt voraus, dass die Monitoring-Nachrichten selbst nicht durch Angreifer manipuliert werden können (CAN-Bus-Spoofing). Eine Erweiterung von ACES um kryptographische Nachrichtenauthentifizierung – etwa durch CAN-SEC gemäß

ISO/SAE 21434 [13] oder durch AUTOSAR SecOC [14] – ist für den praktischen Einsatz unabdingbar. Angreifer könnten gezielt gefälschte Auslastungsinformationen injizieren, um den Scheduler in einen energieineffizienten oder sogar nicht-schedulierbaren Zustand zu treiben. Die formale Analyse solcher Angriffsvektoren im Rahmen der Spieltheorie (Stackelberg-Spiel zwischen Scheduler und Angreifer) ist ein offenes Problem.

8.8 Dynamische Lastbalancierung in Multi-Bus-Architekturen

Moderne Fahrzeuge besitzen mehrere CAN-Busse (Antriebsstrang, Fahrwerk, Komfort, Infotainment), die über Gateways verbunden sind. Eine Erweiterung von ACES auf Multi-Bus-Architekturen mit dynamischer Lastbalancierung – bei der Nachrichten bei Überlast von einem Bus auf einen anderen migriert werden können – stellt ein anspruchsvolles verteiltes Optimierungsproblem dar. Hier bieten sich dezentrale Optimierungsalgorithmen wie ADMM (Alternating Direction Method of Multipliers) [15] an, die eine verteilte Energie-Minimierung ohne zentralen Koordinator ermöglichen.

8.9 Einfluss von EMV und Leitungsparasiten

Die vorliegenden Analysen setzen ideale Busbedingungen voraus. In realen Fahrzeugkabeln treten kapazitive Lasten, induktive Kopplungen und HF-Störungen auf, die das Bit-Timing beeinflussen und effektiv die maximale Bitrate reduzieren. Eine statistische Modellierung der effektiven Bitrate $f_{b,\text{eff}}(t)$ unter realen EMV-Bedingungen und deren Einbeziehung in das ACES-Energiemodell ist notwendig, um die tatsächlichen Energieeinsparungen im Fahrzeug zu quantifizieren. Hierzu bieten sich Messreihen in EMV-Prüfanlagen an.

8.10 Standardisierungsbestrebungen

Angesichts der wachsenden Bedeutung von Energieeffizienz in Fahrzeugnetzwerken – besonders in Elektrofahrzeugen, wo jede Milliwatt-Stunde zählt – sollte die Standardisierung von energiebewusstem CAN-Scheduling in künftigen Revisionen von ISO 11898 oder als eigene ISO-Norm angestrebt werden. CAN in Automation (CiA) [16] als relevantes Normungsgremium sollte frühzeitig in den Standardisierungsprozess eingebunden werden, um Interoperabilität zwischen Herstellern sicherzustellen.

Literaturverzeichnis

- [1] International Organization for Standardization (ISO). *ISO 11898-1:2015 – Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling*. ISO, Genève, 2015.
- [2] International Organization for Standardization (ISO). *ISO 11898-1:2015 Amendment 1 – CAN FD*. ISO, Genève, 2015.
- [3] International Organization for Standardization (ISO). *ISO 11898-1:2024 – CAN XL*. ISO, Genève, 2024.
- [4] S. Onori, L. Serrao und G. Rizzoni. *Hybrid Electric Vehicles – Energy Management Strategies*. Springer Briefs in Control, Automation and Robotics, Springer, London, 2016.

- [5] K. Tindell, A. Burns und A. J. Wellings. “Calculating Controller Area Network (CAN) message response times.” *Control Engineering Practice*, 3(8):1163–1169, 1995.
- [6] NXP Semiconductors. *TJA1044 – High-speed CAN transceiver – Product Data Sheet*. Rev. 3.0, NXP Semiconductors, Eindhoven, 2022.
- [7] S. Bai, J. Z. Kolter und V. Koltun. “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling.” *arXiv:1803.01271*, 2018.
- [8] Vector Informatik GmbH. *CANalyzer – Product Description*. Vector Informatik, Stuttgart, 2023.
- [9] AUTOSAR Consortium. *AUTOSAR Adaptive Platform – Specification of Communication Management*. Release R22-11, AUTOSAR, München, 2022.
- [10] G. Behrmann, A. David und K. G. Larsen. “A tutorial on UPPAAL.” In: *Formal Methods for the Design of Real-Time Systems*, LNCS 3185, Springer, 2004, S. 200–236.
- [11] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, Boston, 2002.
- [12] IEEE Standards Association. *IEEE 802.3bw-2015 – Physical Layer Specifications and Management Parameters for 100 Mb/s Operation over a Single Balanced Twisted Pair Cable (100BASE-T1)*. IEEE, Piscataway, 2015.
- [13] International Organization for Standardization (ISO) / SAE International. *ISO/SAE 21434:2021 – Road vehicles – Cybersecurity engineering*. ISO, Genève, 2021.
- [14] AUTOSAR Consortium. *AUTOSAR – Specification of Secure Onboard Communication (SecOC)*. Release R22-11, AUTOSAR, München, 2022.
- [15] S. Boyd, N. Parikh, E. Chu, B. Peleato und J. Eckstein. “Distributed optimization and statistical learning via the alternating direction method of multipliers.” *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.
- [16] CAN in Automation (CiA). *CiA 601 – CAN FD node and system design – Part 1: General requirements*. CiA, Nürnberg, 2018.
- [17] G. C. Buttazzo. *Hard Real-Time Computing Systems – Predictable Scheduling Algorithms and Applications*. 3rd edition, Springer, New York, 2011.
- [18] H. Kopetz. *Real-Time Systems – Design Principles for Distributed Embedded Applications*. 2nd edition, Springer, New York, 2011.